Full Installation & Setup Manual

ChainCert

Blockchain-Based Credential Issuance and Verification Platform

March 25, 2026 — CAP-2459-IT

TABLE OF CONTENTS

Phase 0: System Identity Architecture (Key Concepts)

Deployment Architecture Note

Phase 1: Environment Setup (WSL2, Ubuntu, Node.js)

Phase 2: Hyperledger Fabric Installation

Phase 3: Chaincode Development Setup

Phase 4: Backend API Setup

Phase 5: Copy Project Files from Windows to Ubuntu

Phase 6: CouchDB Admin Database Setup

Phase 7: Start the Fabric Network

Phase 8: Create Chaincert Admin Identity

Phase 9: DLSU Organization Node Creation

Phase 10: Import DLSU Admin into Backend Wallet

Phase 11: Create Subordinate Issuer (INNOCENCIO)

Phase 12: Create Subordinate Issuer (La Salle Computer Society)

Phase 13: Start Issuer CouchDB Containers

Final System State

Appendix A: Master Copy Commands Reference

PHASE 0: SYSTEM IDENTITY ARCHITECTURE

Before starting the installation, it is critical to distinguish between the different types of identities used in the ChainCert ecosystem. Mixing these up is the most common cause of setup failure.

Identity Breakdown Table

Identity Type	Example	Purpose
System Admin Account	admin@email.com	Login to Web Dashboard (stored in CouchDB)
Fabric Admin Identity	chaincert-admin	Performs Blockchain Operations (e.g., registering new issuers)
Issuer Fabric Identity	delasalleuniversity-admin	Root Authority that signs and issues credentials
Subordinate Identity	delasalleuniversity-subordinate-innocencio	Department authorized to issue under the Root
Subordinate Identity	delasalleuniversity-subordinate-lasallecomputersociety	Department authorized to issue under the Root

Deployment Architecture Note

This installation manual documents the complete setup procedure for Design 1 (Platform Model) — the current ChainCert deployment, where all infrastructure (peers, orderers, CAs, CouchDB instances) is managed and operated by ChainCert on a single server. This is the deployment model used for the prototype and is sufficient to demonstrate the system's full credential issuance, verification, and auditability capabilities.

Moreover, ChainCert's architecture supports two additional deployment models:

1) **Design 2 (Hybrid)**

Issuers deploy their own peer, CA, and CouchDB on their own servers while ChainCert retains governance authority. The key differences from this manual are:

- the issuer executes Phases 9–13 on their own infrastructure using a deployment kit provided by ChainCert
- Connection profiles change from localhost to the issuer's server address,

- gateway discovery as Localhost changes from true to false in the backend configuration.

2) **Design 3 (Full Consortium)**

Same infrastructure as Design 2, but the configtx.yaml Admins policy changes from OR(Org1MSP.admin) to MAJORITY(all member MSPs), requiring channel configuration updates to be signed by multiple existing organizations before a new org can join.

> For detailed technical guidance on Design 2 and Design 3 deployment processes, refer to the Technical Manual (CAP-2459-IT), Section 19: Scalability and Future Architecture.

PHASE 1: ENVIRONMENT SETUP

1.1 Install WSL2

> **Directory:** Windows PowerShell (Run as Administrator)

Command

```
wsl --install
```

What it does

Installs Windows Subsystem for Linux (WSL2) along with the default Ubuntu distribution. This allows you to run a Linux environment directly within Windows.

> **Expected Output**
>
> Installing: Ubuntu
>
> Installation successful. Restart required.

> **Rationale**
>
> Hyperledger Fabric and Docker are designed to run in Linux environments. WSL2 provides a lightweight Linux environment that allows these tools to run without needing a full virtual machine.

Source: OG SETUP

> *Note: If WSL is already installed, this command may return an error such as 'WSL is already installed.' This is expected and can be safely ignored.*

1.2 Set WSL Default Version

> **Directory:** Windows PowerShell (Administrator)

Command

```
wsl --set-default-version 2
```

What it does

Configures WSL so that any installed Linux distribution runs using WSL2, which provides improved virtualization and kernel compatibility.

> **Expected Output**
>
> The operation completed successfully.

> **Rationale**
>
> WSL2 uses a full Linux kernel which is required for Docker container support and Hyperledger Fabric networking.

Source: OG SETUP

1.3 Verify WSL Installation

> **Directory:** Windows PowerShell

Command

```
wsl -l -v
```

What it does

Displays all installed WSL distributions along with their running state and WSL version.

> **Expected Output**
>
> NAME STATE VERSION
>
> Ubuntu Running 2

> **Rationale**

This step confirms that Ubuntu is installed and running under WSL2, ensuring compatibility with Docker and Fabric tools.

Source: OG SETUP

Note: The STATE may show 'Stopped' if the distribution is not currently running. A 'docker-desktop' entry may also appear — this is normal.

1.4 Prepare Ubuntu Environment

Directory: Ubuntu Terminal (WSL)

Command

```
sudo apt update && sudo apt upgrade -y
```

What it does

Updates Ubuntu's package lists and upgrades all installed system packages to the latest versions available.

Expected Output

Reading package lists... Done

Building dependency tree

Calculating upgrade... Done

Rationale

Ensures the Linux environment has up-to-date dependencies and security patches before installing development tools such as Node.js and Hyperledger Fabric.

Source: OG SETUP

Note: Actual output will be significantly more verbose than shown. The key indicator of success is the final line showing upgrade completion with no errors.

1.5 Install Essential Development Tools

Directory: Ubuntu Terminal (WSL)

Command

```
sudo apt install curl wget git -y
```

What it does

Installs essential command-line tools: curl (download scripts), wget (file downloader), git (version control).

Expected Output

Setting up curl...

Setting up git...

Rationale

These utilities are required for downloading Hyperledger Fabric binaries, cloning repositories, and running installation scripts.

Source: OG SETUP

1.6 Install Node.js v18

Directory: Ubuntu Terminal (WSL)

Command

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

What it does

Adds the NodeSource repository and installs Node.js version 18, including npm.

Expected Output

node --version → v18.x.x

npm --version → 9.x.x

Rationale

> Node.js is required for running Fabric chaincode written in JavaScript and running the ChainCert backend API.

Source: OG SETUP

PHASE 2: HYPERLEDGER FABRIC INSTALLATION

2.1 Download Fabric Samples and Binaries

> **Directory:** Ubuntu Terminal (WSL)

Command

```
mkdir -p ~/fabric-samples
cd ~/fabric-samples
curl -sSL https://bit.ly/2ysbOFE | bash -s -- 2.5.0 1.5.5
```

What it does

Downloads the official Hyperledger Fabric samples repository, including Fabric CLI binaries, Docker container images, and example networks (such as test-network).

> **Expected Output**
>
> Installing Hyperledger Fabric binaries
>
> Pulling fabric-peer image
>
> Pulling fabric-orderer image

> **Rationale**
>
> The test-network inside this repository serves as the base blockchain infrastructure for the ChainCert project.

Source: OG SETUP

2.2 Add Fabric Binaries to PATH

> **Directory:** Ubuntu Terminal

Command

```
echo 'export PATH=$PATH:~/fabric-samples/fabric-samples/bin' >> ~/.bashrc
source ~/.bashrc
```

What it does

Adds Hyperledger Fabric CLI binaries to the global system path so commands like peer, configtxgen, and configtxlator can be executed from any directory.

> **Rationale**
>
> Without adding the Fabric binaries to PATH, you would need to manually navigate to the Fabric bin directory before running Fabric commands.

Source: OG SETUP

> *IMPORTANT: Single quotes are required to prevent $PATH from expanding immediately. On WSL2, the Windows PATH contains spaces and parentheses (e.g. Program Files (x86)) that would corrupt .bashrc if expanded. The correct path is fabric-samples/fabric-samples/bin because the bootstrap script clones the repository as a subdirectory.*

PHASE 3: CHAINCODE DEVELOPMENT SETUP

3.1 Create Chaincode Directory

> **Directory:** Ubuntu Terminal

Command

```
mkdir -p ~/chaincert-chaincode
cd ~/chaincert-chaincode
```

What it does

Creates the project directory where the ChainCert smart contract (chaincode) will be stored.

> **Rationale**
>
> Separating the chaincode into its own directory keeps the blockchain logic isolated from the backend API.

Source: OG SETUP

3.2 Initialize Node Project

> **Directory:** ~/chaincert-chaincode

Command

```
npm init -y
```

What it does

Creates a package.json file which manages dependencies for the chaincode project.

> **Rationale**
>
> NodeJS chaincode requires a package.json file to manage dependencies such as Fabric libraries.

Source: OG SETUP

3.3 Install Fabric Chaincode Libraries

> **Directory:** ~/chaincert-chaincode

Command

```
npm install fabric-contract-api fabric-shim
```

What it does

Installs the Fabric libraries required to write and run smart contracts.

> **Rationale**
>
> The fabric-contract-api library allows developers to implement blockchain logic using JavaScript classes.

Source: OG SETUP

PHASE 4: BACKEND API SETUP

4.1 Create Backend Directory

> **Directory:** Ubuntu Terminal

Command

```
mkdir -p ~/chaincert-api
cd ~/chaincert-api
```

What it does

Creates the directory that will contain the ChainCert backend API.

> **Rationale**
>
> This directory hosts the NodeJS backend responsible for interacting with the blockchain network.

Source: OG SETUP

4.2 Initialize Backend Project

> **Directory:** ~/chaincert-api

Command

```
npm init -y
```

What it does

Creates the package.json configuration file for the backend project.

> **Rationale**
>
> The backend API requires dependency management for packages such as Express, Fabric SDK, and authentication libraries.

Source: OG SETUP

4.3 Install Backend Dependencies

> **Directory:** ~/chaincert-api

Command

```
npm install express cors fabric-network fabric-ca-client jsonwebtoken bcryptjs multer dockerode node-cron nodemailer nano dotenv

# Install frontend dependencies (run from Windows project folder)
npm install jspdf qrcode
```

What it does

Installs backend dependencies: Express (REST API server), Fabric SDK (blockchain interaction), JWT (authentication), bcrypt (password hashing), multer (file uploads), dockerode (Docker control), node-cron (credential expiry automation), nodemailer (email OTP), nano (CouchDB client), dotenv (environment variables).

> **Rationale**
>
> These libraries enable the backend API to authenticate users, interact with blockchain, manage document uploads, and control Docker containers. Node-cron allows the system to periodically check for credentials whose expiration date has passed.

Source: COPY COMMANDS / OG SETUP

PHASE 5: COPY PROJECT FILES FROM WINDOWS TO UBUNTU

5.1 Execute Global Copy Commands

> ***NOTE: Refer to Appendix A**: Copy Commands Reference at the end of this manual for the full list of specific commands required to sync your environment.*

> **Directory:** Ubuntu Terminal (WSL)

Command (Example)

```
cp "/mnt/c/Users/<USERNAME>/<PATH_TO_PROJECT>/backend/sdk-server-fixed.js"
~/chaincert-api/sdk-server-fixed.js
```

What it does

Transfers the necessary application and configuration files from the Windows filesystem directly into the corresponding Ubuntu directories.

> **Rationale**
>
> By using the full path starting from /mnt/c/, files can be moved in a single command without needing to change directories first. This ensures all backend, chaincode, and network configuration files are placed in their required runtime locations.

Source: COPY COMMANDS

> ***IMPORTANT**: Before copying issuerRequirements.js:*
>
> - *create the config/ directory first:mkdir -p ~/chaincert-api/config*
> - *Then copy: cp "<PROJECT_ROOT>/backend/issuerRequirements.js" ~/chaincert-api/config/issuerRequirements.js*
> - *The config/ directory does not exist by default. Without it, the cp command fails and the backend will crash when processing issuer registrations.*

IMPORTANT: Shell scripts copied from Windows will have Windows line endings (\r\n) that bash cannot parse. After copying any .sh file, run:sed -i 's/\r//' <filename>.shThis removes the carriage return characters. This is required for registerEnrollFORDLSU.sh and any other shell scripts.

PHASE 6: COUCHDB ADMIN DATABASE SETUP

__NOTE__: Phase 6.3 requires the backend API to be running on port 4000. Start the backend first with:
cd ~/chaincert-api && node sdk-server-fixed.js

6.1 Start CouchDB Container

Directory: Ubuntu Terminal

Command

```
docker run -d \
  --name couchdb-admin \
  -p 5985:5984 \
  -e COUCHDB_USER=admin \
  -e COUCHDB_PASSWORD=adminpw \
  apache/couchdb:latest
```

What it does

Creates a CouchDB container used to store issuer records, user accounts, and notification logs.

Expected Output

Docker returns a container ID indicating successful startup.

Rationale

The backend API uses CouchDB as a database to manage system records separate from the blockchain ledger.

Source: OG SETUP

6.2 Create Required Databases

Directory: Ubuntu Terminal

Command

```
curl -X PUT http://admin:adminpw@localhost:5985/users
curl -X PUT http://admin:adminpw@localhost:5985/issuers
curl -X PUT http://admin:adminpw@localhost:5985/user_notifications
curl -X PUT http://admin:adminpw@localhost:5985/couchdb_connections
curl -X PUT http://admin:adminpw@localhost:5985/profile_share_links
curl -X PUT http://admin:adminpw@localhost:5985/admin_notifications
curl -X PUT http://admin:adminpw@localhost:5985/wallet_user_credentials
```

What it does

Creates all seven required databases for the ChainCert admin CouchDB instance.

Expected Output

CouchDB returns: {"ok":true}

Rationale

These databases store system metadata used by the backend application: users → user accounts and admin accounts; issuers → issuer registration records and status; user_notifications → student/user credential notifications; couchdb_connections → maps issuer IDs to their CouchDB URLs; profile_share_links → share link tokens, expiry, view tracking; admin_notifications → admin dashboard notification events; wallet_user_credentials → user-side credential copies (student wallets).

Source: OG SETUP

6.3 Create System Admin Account

Directory: Ubuntu Terminal (WSL)

Command

```
curl -X POST http://localhost:4000/api/auth/register \
```

```
  -H "Content-Type: application/json" \
  -d '{
    "username": "admin",
    "email": "admin@email.com",
    "password": "adminpw123",
    "role": "admin"
  }'

curl -X POST http://localhost:4000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "admin2",
    "email": "admin2@email.com",
    "password": "adminpw123",
    "role": "admin"
  }'
```

What it does

Registers the initial system administrator accounts used to access the admin dashboard. These accounts are stored in the users CouchDB database and allow administrative operations such as approving issuer registrations, viewing system users, managing credentials, and approving subordinate issuers.

Expected Output

{ "success": true, "message": "User registered successfully" }

Rationale

The system requires at least one administrative account to approve issuer registrations and manage the platform. Without creating this account, no administrator would exist to approve root authorities or subordinate issuers.

Source: PREREQUISITES (Admin Dashboard Setup)

PHASE 7: START THE FABRIC NETWORK

> Before starting the network, verify no conflicting containers are running: docker ps. If containers from a previous session are still running, stop them with: docker stop $(docker ps -aq)

7.1 Navigate to Test Network Directory

> **Directory:** Ubuntu Terminal

Command

```
cd ~/fabric-samples/fabric-samples/test-network
```

What it does

Moves into the Fabric test network directory where network scripts are located.

> **Rationale**
>
> All Fabric network lifecycle commands must be executed from this directory.

Source: NODE CREATION

7.2 Start Fabric Network and Create Channel

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
./network.sh up createChannel -ca -s couchdb -c chaincert-channel
```

What it does

Starts the Fabric blockchain network (peers, orderer, CAs, CouchDB) and creates the chaincert-channel. Docker containers for peer nodes, orderer, certificate authorities, and CouchDB will start.

> **Rationale**

> The Fabric network must be running before smart contracts and organizations can interact with the blockchain.

Source: NODE CREATION

PHASE 8: CREATE CHAINCERT ADMIN IDENTITY

8.1 Set CA Client Home Directory

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
export FABRIC_CA_CLIENT_HOME=${PWD}/organizations/peerOrganizations/org1.example.com/
```

What it does

Sets the Fabric CA client working directory to Org1's MSP directory. This tells the CA client which admin enrollment credentials to use for authenticating the register command. Without this, the CA client will not find valid enrollment information and will fail with 'Enrollment information does not exist.'

> **Rationale**
>
> The fabric-ca-client command needs to know which organization's admin enrollment to use for authentication. Setting FABRIC_CA_CLIENT_HOME points it to Org1's MSP where the admin certificate and private key reside.

Source: PREREQUISITES

8.2 Register Fabric Admin Identity

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
fabric-ca-client register \
  --caname ca-org1 \
```

```
  --id.name chaincertadmin \
  --id.secret chaincertadminpw \
  --id.type client \
  --tls.certfiles "${PWD}/organizations/fabric-ca/org1/ca-cert.pem"
```

What it does

Registers a new identity in the Fabric Certificate Authority.

> **Rationale**
>
> This step defines the admin identity in the CA database. The --tls.certfiles flag is required for TLS-enabled CA communication.

Source: PREREQUISITES

8.3 Enroll Fabric Admin Identity

> **Directory:** ~/chaincert-api

Command

```
node enrollChaincertAdmin.js
```

What it does

Creates the Fabric identity and stores it inside the backend wallet at wallet/chaincert-admin.id.

> **Expected Output**
>
> A file appears inside: wallet/chaincert-admin.id

> **Rationale**
>
> The backend API uses this identity to sign blockchain transactions.

Source: PREREQUISITES

PHASE 9: ROOT AUTHORITY (DLSU) ORGANIZATION NODE CREATION

This phase includes deploying the DLSU Certificate Authority, registering/enrolling the DLSU admin and peer, generating MSP artifacts, creating the organization definition, updating the channel configuration, starting the DLSU peer, joining it to the channel, and installing/approving chaincode.

9.1 Prepare DLSU Directory Structure

> ***IMPORTANT**: At this point, other **DLSU-Related directories were already created** during Phase 5 upon following Appendix A.5*

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
mkdir -p organizations/fabric-ca/dlsu
```

What it does

Creates the directory structure required for DLSU organization files (compose files, CA scripts, etc.).

> **Rationale**
>
> The DLSU node requires dedicated directories for Docker Compose files, CA enrollment scripts, and peer configuration.

Source: NODE CREATION

9.2 Deploy DLSU Certificate Authority

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
docker-compose -f addDLSU/compose/compose-ca-dlsu.yaml up -d
```

What it does

Deploys the DLSU Certificate Authority container.

> **Rationale**
>
> Essential for managing DLSU-specific identities.

Source: NODE CREATION

9.3 Register and Enroll DLSU Admin and Peer

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Step 1 — Fix Windows Line Endings

```
sed -i 's/\r//' addDLSU/fabric-ca/registerEnroll.sh
```

Removes Windows carriage return characters (\r) from the script. Files copied from Windows via /mnt/c/ have \r\n line endings which bash cannot parse.

Step 2 — Run Enrollment Script

```
cd addDLSU/fabric-ca
source registerEnroll.sh createDLSU
```

Registers and enrolls the DLSU admin and peer identities with the DLSU Certificate Authority. Generates the MSP artifacts required for DLSU to participate in the network.

> *IMPORTANT: The script must be run from addDLSU/fabric-ca/ because it uses ../../ relative paths to locate CA certificates and MSP directories. Running from test-network/ will cause 'no such file or directory' errors.*

Source: NODE CREATION

9.4 Copy configtx-WITHDLSU.yaml

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/configtx-WITHDLSU.yaml"
~/fabric-samples/fabric-samples/test-network/configtx/configtx.yaml
```

What it does

Replaces the active channel configuration file with the version that includes DLSU's organization definition. Phase 5 copies configtx-DEFAULT.yaml which only defines Org1 and Org2. The configtxgen command in the next step requires DLSU to be defined in the active configtx.yaml before it can generate DLSUMSP.json.

> *TIMING: configtx-DEFAULT.yaml is used for the initial network startup (Phase 7) with only Org1 and Org2. configtx-WITHDLSU.yaml must be copied here because DLSU must be defined before configtxgen -printOrg DLSUMSP can succeed.*

Source: NODE CREATION

9.5 Generate DLSU Organization Definition

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
cd configtx
export FABRIC_CFG_PATH=$PWD
configtxgen -printOrg DLSUMSP > ../addDLSU/DLSUMSP.json
```

What it does

Generates the DLSU organization definition file (DLSUMSP.json) which is used to add DLSU to the channel configuration.

Source: NODE CREATION

9.6–9.12 Channel Update, Peer Join, and Chaincode

9.6 Fetch and Decode Current Channel Configuration

Directory: ~/fabric-samples/fabric-samples/test-network

Command

```
export FABRIC_CFG_PATH=$PWD/../config
export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

peer channel fetch config config_block.pb -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  -c chaincert-channel --tls --cafile $ORDERER_CA

configtxlator proto_decode --input config_block.pb --type common.Block --output config_block.json
jq .data.data[0].payload.data.config config_block.json > config.json
```

What it does:

Sets the CLI context to Org1 admin, fetches the latest channel configuration block from the orderer, and decodes it into editable JSON format.

9.7 Merge DLSU into Channel Configuration

Directory: ~/fabric-samples/fabric-samples/test-network

Command

```
jq -s '.[0] * {"channel_group":{"groups":{"Application":{"groups":{"DLSUMSP":.[1]}}}}}' config.json
addDLSU/DLSUMSP.json > modified_config.json

configtxlator proto_encode --input config.json --type common.Config --output original_config.pb
configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
configtxlator compute_update --channel_id chaincert-channel --original original_config.pb --updated
modified_config.pb --output config_update.pb
```

What it does:

Inserts DLSU's organization definition into the channel config JSON, encodes both the original and modified configs to protobuf, and computes the delta (only the changes).

9.8 Prepare Channel Update Envelope

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
configtxlator proto_decode --input config_update.pb --type common.ConfigUpdate > config_update.json

jq -n '{payload: {header: {channel_header: {channel_id: "chaincert-channel", type: 2}}, data: {config_update:
(input)}}}' config_update.json > config_update_in_envelope.json

configtxlator proto_encode --input config_update_in_envelope.json --type common.Envelope --output
config_update_in_envelope.pb
```

What it does:

Wraps the channel config delta in an envelope that can be signed and submitted to the orderer.

9.9 Sign and Submit Channel Update

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
# Sign as Org1
```

```
peer channel signconfigtx -f config_update_in_envelope.pb

# Switch to Org2 admin and sign
export CORE_PEER_LOCALMSPID="Org2MSP"
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2
.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.or
g2.example.com/tls/ca.crt
export CORE_PEER_ADDRESS=localhost:9051

peer channel signconfigtx -f config_update_in_envelope.pb

# Switch back to Org1 and submit
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1
.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.or
g1.example.com/tls/ca.crt
export CORE_PEER_ADDRESS=localhost:7051

peer channel update -f config_update_in_envelope.pb -c chaincert-channel -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA
```

What it does:

Both Org1 and Org2 cryptographically sign the channel update (satisfying the channel's modification policy), then Org1 submits it to the orderer. After this, DLSU is officially recognized as a channel member.

9.10 Start DLSU Peer Container

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
# Switch to DLSU admin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.example.com/msp
export CORE_PEER_ADDRESS=localhost:12051

# Start DLSU peer
cd addDLSU/compose
docker-compose -f compose-dlsu.yaml up -d
cd ../..
```

What it does:

Sets the CLI context to DLSU admin and starts the DLSU peer container

9.11 Join DLSU Peer to Channel

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
peer channel fetch 0 channel-artifacts/chaincert-channel.block -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  -c chaincert-channel --tls --cafile \
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"

docker exec \
  -e CORE_PEER_TLS_ENABLED=true \
  -e CORE_PEER_LOCALMSPID="DLSUMSP" \
  -e
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt \
```

```
  -e
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizatio
ns/dlsu.example.com/users/Admin@dlsu.example.com/msp \
  -e CORE_PEER_ADDRESS=localhost:12051 \
  peer0.dlsu.example.com \
  peer channel join -b channel-artifacts/chaincert-channel.block

# Verify if part na sa peer ng chaincert-channel
docker exec peer0.dlsu.example.com peer channel list
```

What it does:

Fetches the channel genesis block and runs the join command inside the DLSU peer container so it officially joins chaincert-channel and starts syncing the ledger. After, verify if successful.

9.12 Deploy and Approve Chaincode for DLSU

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
# Deploy chaincode to the network (Org1 and Org2)
./network.sh deployCC \
  -c chaincert-channel \
  -ccn chaincert \
  -ccl javascript \
  -ccp ~/chaincert-chaincode \
  -ccv 1.0

# As DLSU admin, install the chaincode
peer lifecycle chaincode install chaincert.tar.gz

# Get the package ID (copy the full ID from the output)
peer lifecycle chaincode queryinstalled

# Approve as DLSU (replace package-id with actual value from queryinstalled)
peer lifecycle chaincode approveformyorg -o localhost:7050 \
```

```
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID chaincert-channel \
  --name chaincert \
  --version 1.0 \
  --package-id <PASTE_PACKAGE_ID_HERE> \
  --sequence 1 \
  --tls \
  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"

# Verify all 3 orgs approved
peer lifecycle chaincode querycommitted --channelID chaincert-channel --name chaincert
```

What it does:

Deploys chaincode to the network (Org1 and Org2 automatically), installs it on DLSU's peer, and DLSU approves it. The final verify command should show Approvals: [DLSUMSP: true, Org1MSP: true, Org2MSP: true].

> ***Note**: The --package-id value is unique to each installation. Copy the exact value from the queryinstalled output — it looks like chaincert_1.0:1933927ebeca046496a17254a40fb15b58a36e09238099b40c49138d83232a19.*

PHASE 10: IMPORT DLSU ADMIN INTO BACKEND WALLET

10.1 Import DLSU Wallet Identity

> **Directory:** ~/chaincert-api

Command

```
node setupWallet_DLSU.js
```

What it does

Reads the DLSU admin certificate and private key from the MSP directory and imports them into the backend wallet as delasalleuniversity-admin with mspId DLSUMSP.

Expected Output

A file appears inside: wallet/delasalleuniversity-admin.id

Rationale

The backend needs this wallet identity to submit blockchain transactions on behalf of DLSU (credential issuance, template creation, etc.).

Source: NODE CREATION

PHASE 11: CREATE SUBORDINATE ISSUER (INNOCENCIO)

11.1 Set DLSU CA Context

Directory: ~/fabric-samples/fabric-samples/test-network

Command

```
export FABRIC_CA_CLIENT_HOME=${PWD}/organizations/peerOrganizations/dlsu.example.com/
```

What it does

Sets the CA client context to DLSU's organization so that subordinate identities are registered under DLSU's Certificate Authority.

Source: Manual subordinate workflow

11.2 Register Subordinate Identity

Directory: ~/fabric-samples/fabric-samples/test-network

Command

```
fabric-ca-client register \
  --caname ca-dlsu \
  --id.name delasalleuniversity-subordinate-innocencio \
  --id.secret innocenciopw \
  --id.type client \
  --tls.certfiles "${PWD}/organizations/fabric-ca/dlsu/ca-cert.pem"
```

What it does

Registers the INNOCENCIO subordinate identity in DLSU's Certificate Authority. The identity type is 'client' because subordinates only need to submit transactions, not administer the peer or channel.

Source: Manual subordinate workflow

11.3 Enroll Subordinate Identity

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
fabric-ca-client enroll \
  -u https://delasalleuniversity-subordinate-innocencio:innocenciopw@localhost:12054 \
  --caname ca-dlsu \
  -M
"${PWD}/organizations/peerOrganizations/dlsu.example.com/users/delasalleuniversity-subordinate-innocencio@dlsu.
example.com/msp" \
  --tls.certfiles "${PWD}/organizations/fabric-ca/dlsu/ca-cert.pem"
```

What it does

Enrolls the registered identity, generating a certificate and private key pair. The MSP artifacts are stored in the DLSU organization's users directory.

Source: Manual subordinate workflow

11.4 Copy NodeOU Configuration

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
cp "${PWD}/organizations/peerOrganizations/dlsu.example.com/msp/config.yaml" \

"${PWD}/organizations/peerOrganizations/dlsu.example.com/users/delasalleuniversity-subordinate-innocencio@dlsu.example.com/msp/config.yaml"
```

What it does

Copies the NodeOU configuration to the subordinate's MSP directory. This configuration file tells Fabric how to classify identities (admin, client, peer, orderer) based on their OU attributes.

Source: Manual subordinate workflow

11.5 Import Subordinate Wallet Identity

> **Directory:** ~/chaincert-api

Command

```
node setup_Wallet_DLSU_Subordinate_INNOCENCIO.js
```

What it does

Reads the INNOCENCIO certificate and private key from the MSP directory and imports them into the backend wallet with mspId DLSUMSP.

> **Expected Output**
>
> A file appears inside: wallet/delasalleuniversity-subordinate-innocencio.id

> **Rationale**
>
> The backend needs this wallet identity to submit blockchain transactions on behalf of INNOCENCIO.

Source: Manual subordinate workflow

11.6 Start Subordinate CouchDB Container

> **Directory:** ~/fabric-samples/fabric-samples/test-network/addDLSU/compose

Command

```
docker-compose -f compose-couch-innocencio.yaml up -d
```

What it does

Starts the CouchDB container dedicated to the INNOCENCIO subordinate issuer on port 9085.

> **Rationale**
>
> Each issuer has its own CouchDB container to isolate credential storage per organization.

Source: Manual subordinate workflow

11.7 Create Subordinate CouchDB Databases

Command

```
curl -X PUT http://admin:adminpw@localhost:9085/wallet_issuer_credentials
curl -X PUT http://admin:adminpw@localhost:9085/issued_credentials
```

What it does

Creates the two required databases for the INNOCENCIO issuer.

> **Expected Output**
>
> {"ok":true}

> **Rationale**
>
> wallet_issuer_credentials stores credential templates created by the issuer. issued_credentials stores issued credential instances.

Source: Manual subordinate workflow

11.8 Register couchdb_connections Entry

Command

```
curl -X POST http://admin:adminpw@localhost:5985/couchdb_connections \
  -H "Content-Type: application/json" \
  -d '{
    "orgName": "innocencio",
    "couchDBUrl": "http://admin:adminpw@localhost:9085"
  }'
```

What it does

Registers the INNOCENCIO CouchDB connection in the admin database so the backend can dynamically resolve the correct CouchDB instance during issuer approval and credential operations.

Expected Output

{"ok":true, "id":"...", "rev":"..."}

Rationale

The backend uses this mapping to dynamically connect to the correct issuer CouchDB when issuing credentials, running integrity checks, or loading issuer data.

Source: Manual subordinate workflow

IMPORTANT: orgName must be lowercase. The backend normalizes orgName to lowercase before querying couchdb_connections. Using uppercase (e.g., INNOCENCIO) will cause the approval process to fail with 'No CouchDB connection found.'

PHASE 12: CREATE SUBORDINATE ISSUER (LA SALLE COMPUTER SOCIETY)

12.1 Set DLSU CA Context

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
export FABRIC_CA_CLIENT_HOME=${PWD}/organizations/peerOrganizations/dlsu.example.com/
```

What it does

Sets the CA client context to DLSU's organization for subordinate registration.

Source: Manual subordinate workflow

12.2 Register Subordinate Identity

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
fabric-ca-client register \
  --caname ca-dlsu \
  --id.name delasalleuniversity-subordinate-lasallecomputersociety \
  --id.secret lscspw \
  --id.type client \
  --tls.certfiles "${PWD}/organizations/fabric-ca/dlsu/ca-cert.pem"
```

What it does

Registers the La Salle Computer Society subordinate identity in DLSU's Certificate Authority.

Source: Manual subordinate workflow

12.3 Enroll Subordinate Identity

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
fabric-ca-client enroll \
  -u https://delasalleuniversity-subordinate-lasallecomputersociety:lscspw@localhost:12054 \
  --caname ca-dlsu \
  -M
"${PWD}/organizations/peerOrganizations/dlsu.example.com/users/delasalleuniversity-subordinate-lasallecomputersociety@dlsu.example.com/msp" \
  --tls.certfiles "${PWD}/organizations/fabric-ca/dlsu/ca-cert.pem"
```

What it does

Enrolls the registered identity, generating certificate and private key pair.

Source: Manual subordinate workflow

12.4 Copy NodeOU Configuration

> **Directory:** ~/fabric-samples/fabric-samples/test-network

Command

```
cp "${PWD}/organizations/peerOrganizations/dlsu.example.com/msp/config.yaml" \

"${PWD}/organizations/peerOrganizations/dlsu.example.com/users/delasalleuniversity-subordinate-lasallecomputersociety@dlsu.example.com/msp/config.yaml"
```

What it does

Copies the NodeOU configuration to the subordinate's MSP directory.

Source: Manual subordinate workflow

12.5 Import Subordinate Wallet Identity

> **Directory:** ~/chaincert-api

Command

```
node setup_Wallet_DLSU_Subordinate_LASALLECOMPUTERSOCIETY.js
```

What it does

Imports the La Salle Computer Society wallet identity into the backend.

> **Expected Output**
>
> A file appears inside: wallet/delasalleuniversity-subordinate-lasallecomputersociety.id

Source: Manual subordinate workflow

12.6 Start Subordinate CouchDB Container

> **Directory:** ~/fabric-samples/fabric-samples/test-network/addDLSU/compose

Command

```
docker-compose -f compose-couch-lasallecomputersociety.yaml up -d
```

What it does

Starts the CouchDB container dedicated to the La Salle Computer Society subordinate issuer on port 9086.

> **Rationale**
>
> Each issuer has its own CouchDB container to isolate credential storage per organization.

Source: Manual subordinate workflow

12.7 Create Subordinate CouchDB Databases

Command

```
curl -X PUT http://admin:adminpw@localhost:9086/wallet_issuer_credentials
curl -X PUT http://admin:adminpw@localhost:9086/issued_credentials
```

What it does

Creates the two required databases for the La Salle Computer Society issuer.

> **Expected Output**
>
> {"ok":true}

Source: Manual subordinate workflow

12.8 Register couchdb_connections Entry

Command

```
curl -X POST http://admin:adminpw@localhost:5985/couchdb_connections \
  -H "Content-Type: application/json" \
  -d '{
    "orgName": "lasallecomputersociety",
    "couchDBUrl": "http://admin:adminpw@localhost:9086"
  }'
```

What it does

Registers the La Salle Computer Society CouchDB connection in the admin database.

> **Expected Output**
>
> {"ok":true, "id":"...", "rev":"..."}

Source: Manual subordinate workflow

PHASE 13: START ISSUER COUCHDB CONTAINERS

13.1 Start DLSU CouchDB Container

Directory: ~/fabric-samples/fabric-samples/test-network/addDLSU/compose

Command

```
docker-compose -f compose-couch-dlsu.yaml up -d
```

What it does

Starts the CouchDB container dedicated to the DLSU root authority issuer on port 9084.

Rationale

The root authority issuer needs its own CouchDB container to store templates and issued credentials.

Source: NODE CREATION

13.2 Create DLSU Issuer Databases

Command

```
curl -X PUT http://admin:adminpw@localhost:9084/wallet_issuer_credentials
curl -X PUT http://admin:adminpw@localhost:9084/issued_credentials
```

What it does

Creates the two required databases for the DLSU issuer CouchDB instance.

Expected Output

{"ok":true}

Rationale

wallet_issuer_credentials stores credential templates created by DLSU. issued_credentials stores issued credential instances.

Source: NODE CREATION

13.2A Register DLSU couchdb_connections Entry

Command

```
curl -X POST http://admin:adminpw@localhost:5985/couchdb_connections \
  -H "Content-Type: application/json" \
  -d '{
    "orgName": "delasalleuniversity",
    "couchDBUrl": "http://admin:adminpw@localhost:9084"
  }'
```

Expected Output

{"ok":true, "id":"...", "rev":"..."}

Rationale

Maps the DLSU organization name to its CouchDB URL so the backend can dynamically resolve the correct database during issuer approval and credential operations.

Source: NODE CREATION

FINAL SYSTEM STATE

After completing all phases, the following components should be confirmed operational:

Component	Port	Status
peer0.org1.example.com	7051	✅ Running
peer0.org2.example.com	9051	✅ Running
peer0.dlsu.example.com	12051	✅ Running
orderer.example.com	7050	✅ Running
ca_org1 / ca_org2 / ca_orderer / ca_dlsu	7054 / 8054 / 9054 / 12054	✅ Running
couchdb-admin	5985	✅ Running
couchdb_dlsu (issuer)	9084	✅ Running
couchdb_innocencio (issuer)	9085	✅ Running
couchdb_lasallecomputersociety (issuer)	9086	✅ Running
Chaincode (chaincert v1.0)	—	✅ Deployed (all 3 orgs approved)
chaincert-admin.id	—	✅ In wallet
delasalleuniversity-admin.id	—	✅ In wallet
delasalleuniversity-subordinate-innocencio.id	—	✅ In wallet
delasalleuniversity-subordinate-lasallecomputersociety.id	—	✅ In wallet
couchdb_connections (3 entries)	—	✅ Registered
Backend API	4000	✅ Running
Credential expiry cron job	—	✅ Scheduled

APPENDIX A: MASTER COPY COMMANDS REFERENCE

All copy commands use the pattern: cp "/mnt/c/Users/<USERNAME>/<PATH_TO_PROJECT>/..." ~/target/path. Replace <USERNAME> and <PATH_TO_PROJECT> with your actual Windows username and project folder path.

A.1 Chaincode Files

```
cp "<PROJECT_ROOT>/chaincode/index.js" ~/chaincert-chaincode/index.js
cp "<PROJECT_ROOT>/chaincode/package.json" ~/chaincert-chaincode/package.json
```

A.2 Backend API Core Files

> ***IMPORTANT**: Create the config/ directory before copying issuerRequirements.js:mkdir -p ~/chaincert-api/config*

```
mkdir -p ~/chaincert-api/config
cp "<PROJECT_ROOT>/backend/sdk-server-fixed.js" ~/chaincert-api/sdk-server-fixed.js
cp "<PROJECT_ROOT>/backend/couchDB_service.js" ~/chaincert-api/couchDB_service.js
cp "<PROJECT_ROOT>/backend/.env" ~/chaincert-api/.env
cp "<PROJECT_ROOT>/backend/enrollChaincertAdmin.js" ~/chaincert-api/enrollChaincertAdmin.js
cp "<PROJECT_ROOT>/backend/setupWallet_DLSU.js" ~/chaincert-api/setupWallet_DLSU.js
cp "<PROJECT_ROOT>/backend/setupWallet_ORG1.js" ~/chaincert-api/setupWallet_ORG1.js
cp "<PROJECT_ROOT>/backend/setupWallet_ORG2.js" ~/chaincert-api/setupWallet_ORG2.js
cp "<PROJECT_ROOT>/backend/setup_Wallet_DLSU_Subordinate_INNOCENCIO.js"
~/chaincert-api/setup_Wallet_DLSU_Subordinate_INNOCENCIO.js
cp "<PROJECT_ROOT>/backend/setup_Wallet_DLSU_Subordinate_LASALLECOMPUTERSOCIETY.js"
~/chaincert-api/setup_Wallet_DLSU_Subordinate_LASALLECOMPUTERSOCIETY.js
cp "<PROJECT_ROOT>/backend/issuerRequirements.js" ~/chaincert-api/config/issuerRequirements.js
```

A.3 Controllers

> ***IMPORTANT**: Create the controllers/ directory before copying controller files:mkdir -p ~/chaincert-api/controllers*

```
mkdir -p ~/chaincert-api/controllers
cp "<PROJECT_ROOT>/backend/controllers/authController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/accountController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/issuerCredentialController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/issuerOnboardingController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/adminIssuerController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/adminAuditController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/adminDashboardController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/rootAuthorityController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/walletController.js" ~/chaincert-api/controllers/
cp "<PROJECT_ROOT>/backend/controllers/verifierController.js" ~/chaincert-api/controllers/
```

A.4 Routes

> ***IMPORTANT**: Create the routes/ directory before copying route files:mkdir -p ~/chaincert-api/routes*

```
mkdir -p ~/chaincert-api/routes
cp "<PROJECT_ROOT>/backend/routes/authRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/adminRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/issuerRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/credentialRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/walletRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/verificationRoutes.js" ~/chaincert-api/routes/
cp "<PROJECT_ROOT>/backend/routes/systemRoutes.js" ~/chaincert-api/routes/
```

A.5 Fabric Network Configuration Files

***IMPORTANT**: Create the routes/ directory before copying route files:mkdir -p ~/chaincert-api/routes*

```
mkdir -p ~/fabric-samples/fabric-samples/test-network/addDLSU/compose/docker/peercfg
mkdir -p ~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca
mkdir -p ~/fabric-samples/fabric-samples/test-network/addDLSU/peercfg

cp "<PROJECT_ROOT>/backend/node_creation/dlsu/configtx-DEFAULT.yaml"
~/fabric-samples/fabric-samples/test-network/configtx/configtx.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/configtx-WITHDLSU.yaml"
~/fabric-samples/fabric-samples/test-network/configtx/configtx-WITHDLSU.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/compose/compose-dlsu.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/compose-dlsu.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/compose/compose-ca-dlsu.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/compose-ca-dlsu.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/compose/compose-couch-dlsu.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/compose-couch-dlsu.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/compose/compose-couch-innocencio.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/compose-couch-innocencio.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/compose/compose-couch-lasallecomputersociety.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/compose-couch-lasallecomputersociety.yaml
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/fabric-ca/registerEnroll.sh"
~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca/registerEnroll.sh
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/core.yaml"
~/fabric-samples/fabric-samples/test-network/addDLSU/core.yaml
cp "<PROJECT_ROOT>/backend/connection-delasalleuniversity.json"
~/chaincert-api/connection-delasalleuniversity.json
```

NOTE: After copying configtx-DEFAULT.yaml to configtx/configtx.yaml in Phase 5, you will later need to copy configtx-WITHDLSU.yaml over it in Phase 9.4 before generating DLSUMSP.json. Keep both files available.

NOTE: After copying shell scripts from Windows, fix line endings:sed -i 's/\r//' ~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca/registerEnroll.sh